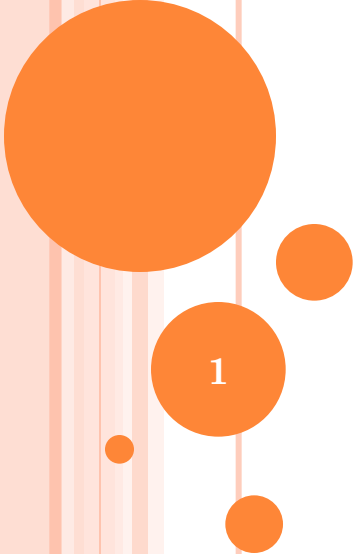




SISTEME DE OPERARE

Curs 7: Planificarea execuției

Kinga Márton

**Universitatea Tehnică din Cluj-Napoca
Departamentul de Calculatoare**

1

PLANIFICAREA EXECUȚIEI

- În cuprins
 - Concepte generale
 - Ciclul de viață al proceselor / threadurilor
 - Planificator vs. Dispecer
 - Planificare preemptivă vs. nonpreemptivă
 - Metrici și obiective de planificare
 - Algoritmi de planificare
 - Exerciții

INTRODUCERE

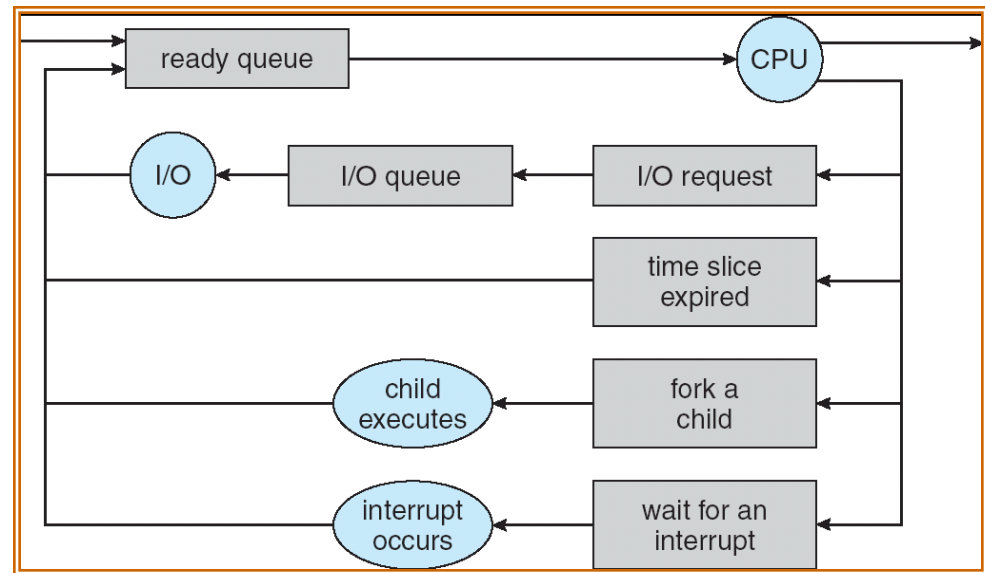
- Resursele majore ale sistemului necesită planificare
 - Mai multe procese / threaduri cer acces concurent la resurse
 - Procesorul, discul, memoria, etc.
- Planificare
 - **Politică de alegere** a procesului / threadului căruia i se atribuie resursa sistem cerută
 - Urmărește optimizarea unor caracteristici ale sistemului
 - Performanță, timp de răspuns, debit, etc.

INTRODUCERE

- Planificarea execuției este **esențială** în SO cu suport pentru **multitasking**
 - Prin comutarea între procese SO poate maximiza utilizarea procesorului -> sistem mai productiv
 - Un proces este executat până în momentul în care trebuie să aștepte după un eveniment blocant – operație de i/e – sau îi expiră cuanta de timp
 - Sau apare un proces mai prioritar
 - Atunci planificatorul alege alt proces spre execuție – din cele menținute în memorie și în starea Ready - conform unui **algoritm de planificare**
- Planificarea proceselor / **threadurilor**
 - Pe SO care suportă threadurile kernel – execuția se efectuează la nivel de thread -> deci de fapt threadurile sunt planificate

INTRODUCERE

- Ciclul de viață al unui proces / thread



- Diferitele stări

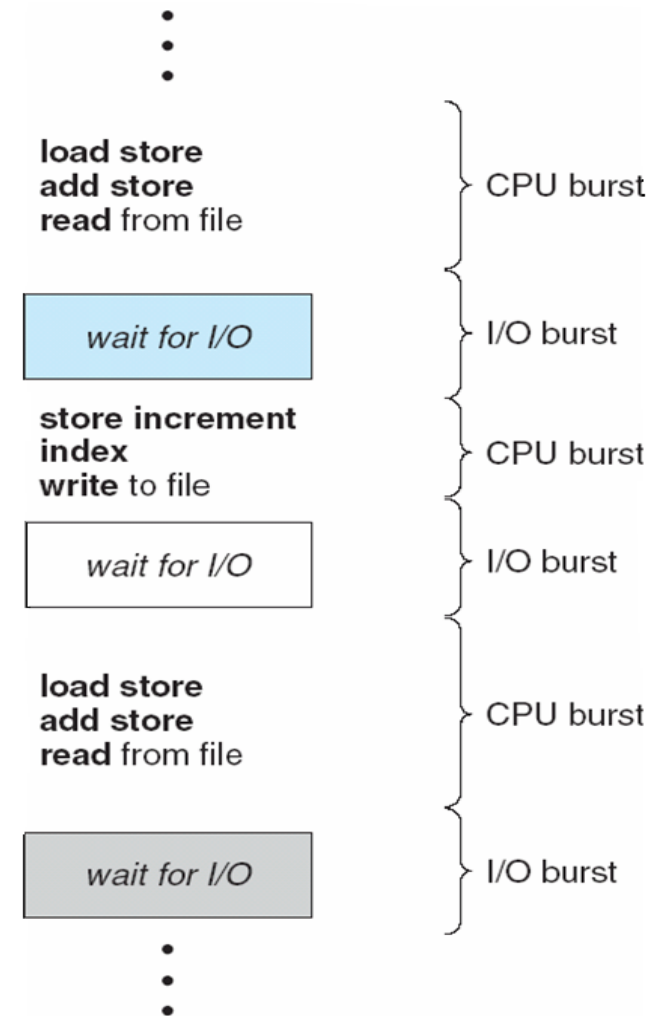
- cozi de așteptare
- conțin BCP / BCT-uri

- Cum decide SO care proces / thread trebuie scos din cozi ?

- accent pe **Ready**
 - Care proces va fi rulat următorul?
 - Cum e implementată coada Ready?
 - Coadă FIFO, arbore, coadă de priorități, listă înlanțuită

INTRODUCERE

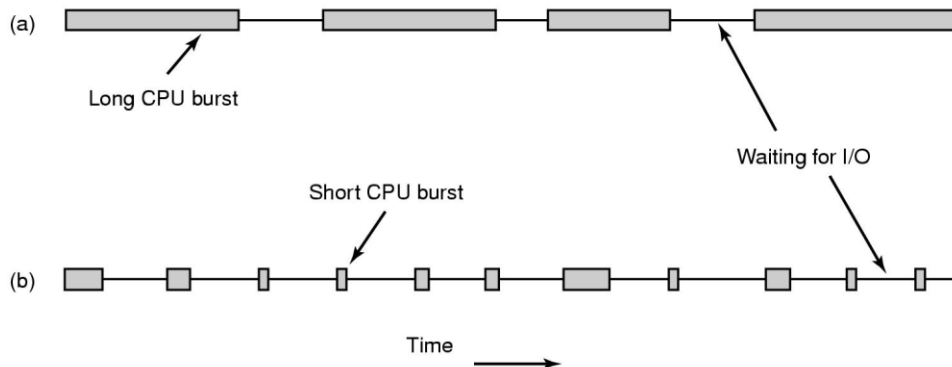
- Execuția proceselor / threadurilor
 - constă din rafală de calcule (cicluri de procesor) și rafală de I/E (cereri de op. I/E - așteptări)
- Categoriile de procese
 - Procese care folosesc preponderent procesorul – intens computaționale
 - Procese CPU-bound
 - Procese care folosesc preponderent operații de I/E
 - Procese I/O-bound



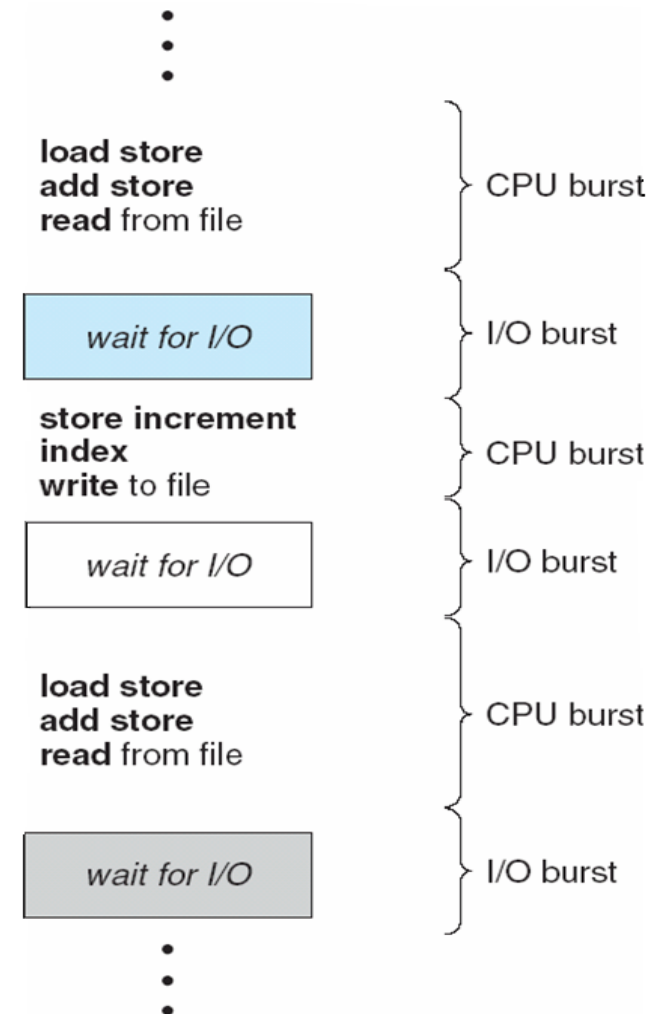
Silberschatz, pg. 184

INTRODUCERE

- Execuția proceselor / threadurilor
 - constă din **rafală de calcule** (cicluri de procesor) și **rafală de I/E** (cereri de op. I/E - așteptări)
- Categoriile de procese
 - a) Procese care folosesc **preponderent procesorul** – intens computaționale
 - Procese CPU-bound
 - b) Procese care folosesc **preponderent operații de I/E**
 - Procese I/O-bound



Tanenbaum, pg. 134



Silberschatz, pg. 184

PLANIFICARE - MECANISME ȘI POLITICI

○ Planificator

- Componentă a SO care **decide** care proces va fi rulat și pentru cât timp
 - **Politică** de nivel superior
 - Planificatorul procesorului
 - Planificare **pe termen scurt**

○ Dispecer

- Modulul care **efectuează schimbarea de context**
 - Predă controlul procesorului procesului/threadului selectat de planificator
- **Mecanism** de nivel coborât
- Invocat la fiecare schimbare de context
 - Trebuie să fie eficient
- Latența dispecerului
 - Timpul necesar pentru suspendarea procesului curent și începerea procesului următor

PLANIFICAREA EXECUȚIEI

○ Planificare

- Termen lung (controlează gradul de multiprogramare)
- Termen mediu (decizii legate de swapping)
- **Termen scurt** - Ready -> Running

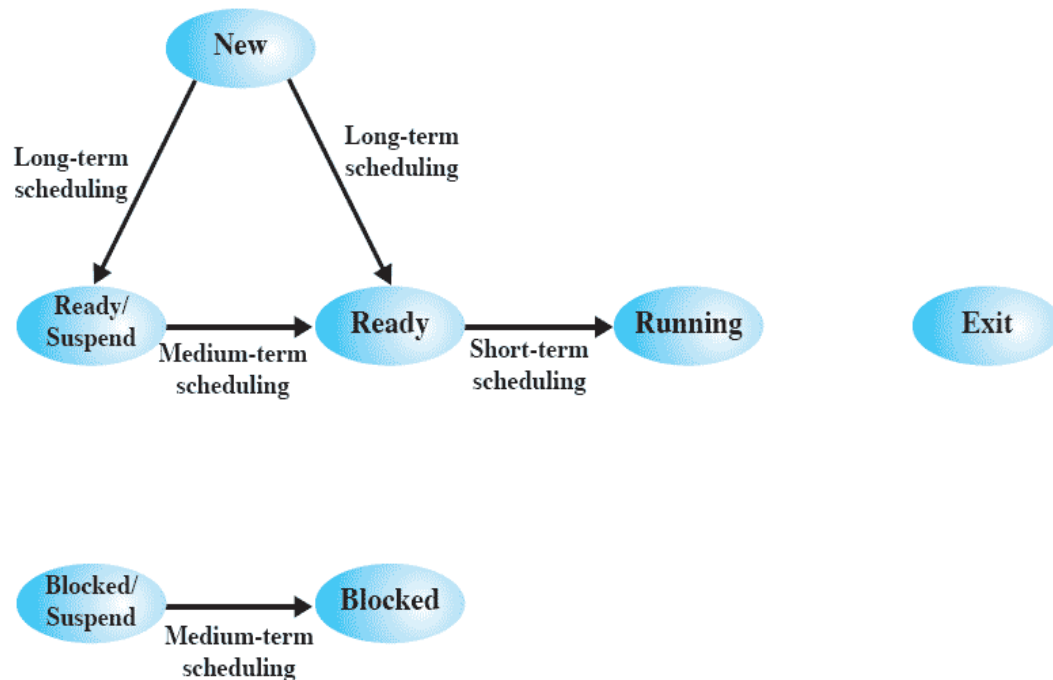


Figure 9.1 Scheduling and Process State Transitions

PLANIFICARE

○ Când se iau decizii de planificare

1. Când un proces trece din starea **În execuție** (**Running**) în starea de **Așteptare** (**Waiting**) (ex. pentru op. I/E, sau wait pt proces fiu)
2. Când un proces trece din starea **În execuție** (**Running**) în starea **Gata de execuție** (**Ready**) (ex. apare o întrerupere, expiră cuanta)
3. Când un proces trece de la starea de **Așteptare** (**Waiting**) la starea **Gata de execuție** (**Ready**) (ex. la terminarea operației de I/E)
4. Când un proces se **termină**

○ Planificare **non-preemptivă** sau **cooperativă** : 1 și 4

- Procesul rulează până când se termină, sau trece în starea de așteptare pt un eveniment blocant – cedează de bună voie procesorul

○ Planificare **preemptivă** : 2 și 3

- Un proces rulează o cantă de timp apoi este suspendat și se alege alt proces – este necesară o întrerupere de ceas

CRITERII / OBIECTIVE / METRICI DE PLANIFICARE

- Utilizarea procesorului - max
- Debit (Throughput) - max
 - Număr de procese executate / unitate de timp
 - Pt. procese lungi poate fi 1 proces / ora
 - Pt procese scurte poate fi 10 procese / sec.
- Timp de așteptare (Waiting time) - min
 - Suma perioadelor petrecute în coada de așteptare a proceselor Ready
- Timp de răspuns (Response time) - min
 - Intervalul de timp dintre momentul lansării jobului până la primul răspuns
 - Importanță mare în sisteme interactive

CRITERII / OBIECTIVE / METRICI DE PLANIFICARE

- **Timp de deservire** (Turnaround time) - min
 - Timpul de la lansarea procesului și până la terminarea acestuia
- **Corectitudine** (Fairness) - max
 - Partajarea procesorului între utilizatori / procese în mod echitabil
 - Procese echivalente să primească timp echivalent pe procesor
- Cele două metrici sunt adesea în contradicție
 - Se poate optimiza performanța cu costul de a preveni rularea unor procese

ALGORITMI DE PLANIFICARE

○ Ipoteze de simplificare

- Pt. înțelegere mai ușoară și introducere gradată în domeniu
- Unele mai nerealiste decât celelalte – dar le vom relaxa gradat

- **Durata de execuție este aceeași pentru fiecare job**
- **Toate joburile sosesc în același moment în sistem**
- **Fiecare job folosește doar procesorul**
- **Durata de execuție este cunoscută în prealabil**

○ Pt început folosim o singură metrică pentru evaluare

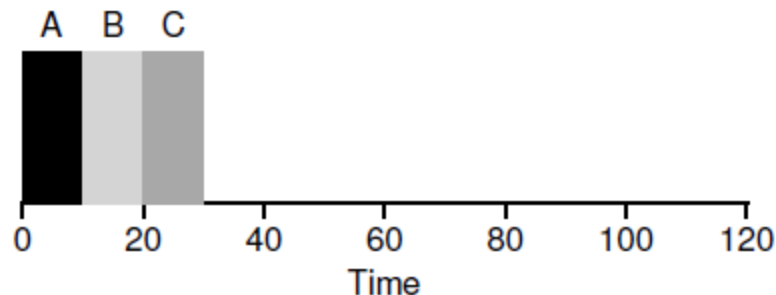
$$T_{turnaround} = T_{completion} - T_{arrival}$$

- Vedem pe parcurs cum se împacă cu corectitudinea și cu timpul de răspuns

ALGORITMUL FIFO / FCFS

○ Firs-In First-Out sau First Come, First Served

- Extrem de simplu și ușor de implementat
- Fiind date ipotezele – funcționează foarte bine
- Algoritm **non-preemptiv**



○ Trei joburi: A, B, C toate ajung deodată $T_{arrival} = 0$

- A un pic mai repede decât B, B un pic mai repede decât C

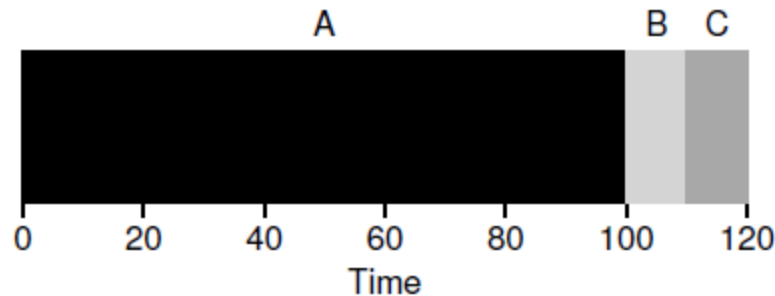
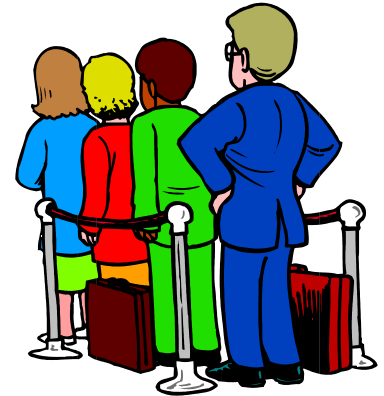
○ Fiecare se execută 10 secunde

○ Timpul de deservire mediu $\frac{10+20+30}{3} = 20$

ALGORITMUL FIFO / FCFS

- Relaxăm o ipoteză → **durată diferită pentru joburi**

- ~~Durata de execuție este aceeași pentru fiecare job~~
- Toate joburile sosesc în același moment în sistem
- Fiecare job folosește doar procesorul
- Durata de execuție este cunoscută în prealabil

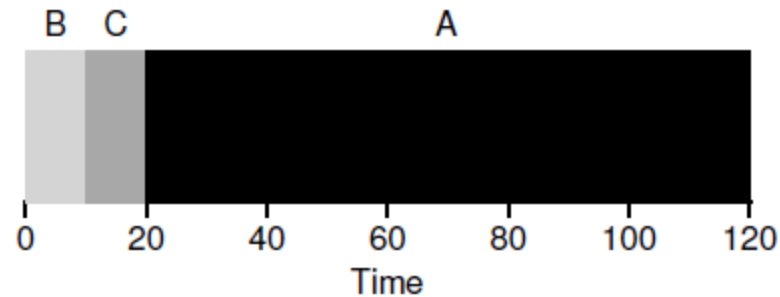


- A se execută 100 secunde, B: 10 sec, C: 10 sec
- Timpul de execuție mediu $\left(\frac{100+110+120}{3} = 110 \right)$
- Efectul de **convoi** – analogia cu statul la coadă

ALGORITMUL SJF

○ Shortest Job First

- Se alege cel mai scurt job pentru execuție
- Algoritm **non-preemptiv**



○ Politică de planificare optimă (până acum!!)

- În contextul dat de ipoteze

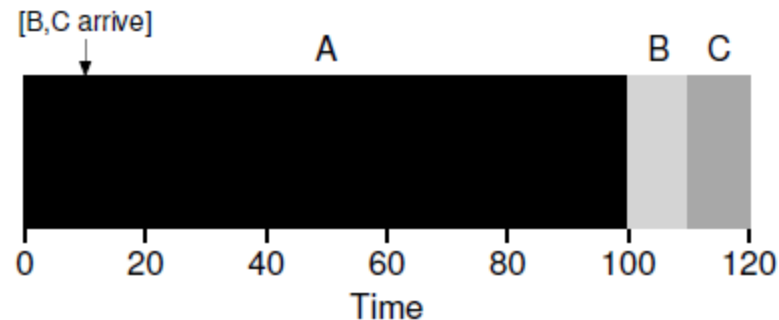
○ Timpul de deservire mediu

- Față de 110 (ex. anterior)

$$\frac{10+20+120}{3} = 50$$

ALGORITMUL SJF

- Relaxăm încă o ipoteză → **joburile sosesc la momente diferite**
 - ~~Durata de execuție este aceeași pentru fiecare job~~
 - ~~Toate joburile sosesc în același moment în sistem~~
 - Fiecare job folosește doar procesorul
 - Durata de execuție este cunoscută în prealabil



- A ajunge primul, iar după 10 secunde B și C
 - Efectul de convoi

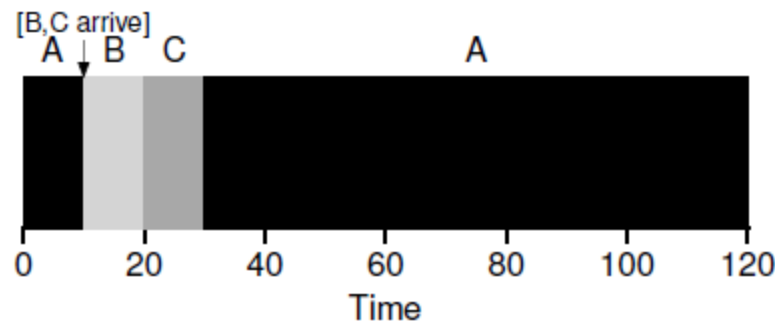
- Timpul de execuție mediu $\frac{100 + (110 - 10) + (120 - 10)}{3} = 103.33$

ALGORITMUL STCF

◦ Shortest Time-to-Completion First

◦ Algoritm preemptiv

- Suspendă temporar execuția unui job și lansează (reia) altul în execuție
- Efectuează schimbări de context
- De fapt este SJF + mecanism de preempțiune
 - Când apare un nou job se decide care va fi selectat spre execuție



- A ajunge primul, apoi B și C
- Când ajunge B și C, se alege B, apoi C, apoi se reia A

- Timpul de deservire mediu $\frac{(120-0)+(20-10)+(30-10)}{3} = 50$

ALGORITMI DE PLANIFICARE

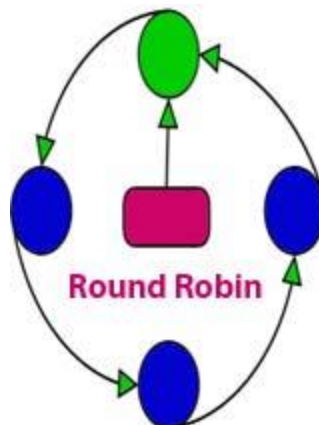
- Algoritmii de până acum sunt potriviți pt sisteme batch
- Introducerea sistemelor time-sharing (de partajare a timpului) a schimbat totul
 - Sisteme interactive
 - Utilizatorul așteaptă răspuns
- Apare o nouă metrică: **Timpul de răspuns**
 - Durata dintre lansarea jobului în execuție și momentul în care este ales (planificat) spre execuție

$$T_{response} = T_{firstrun} - T_{arrival}$$

ALGORITMUL RR

○ Round Robin

- Fiecare job rulează o **cuantă de timp** (time-slice) după care se trece la următorul job
 - în mod repetat
- Algoritm preemptiv
- Joburile cu starea Ready stau într-o coadă de așteptare FIFO



ALGORITMUL RR

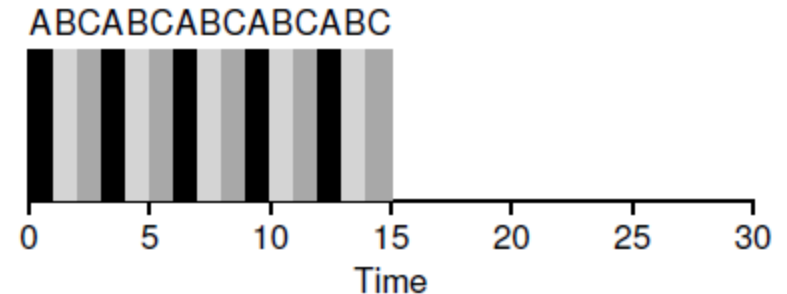
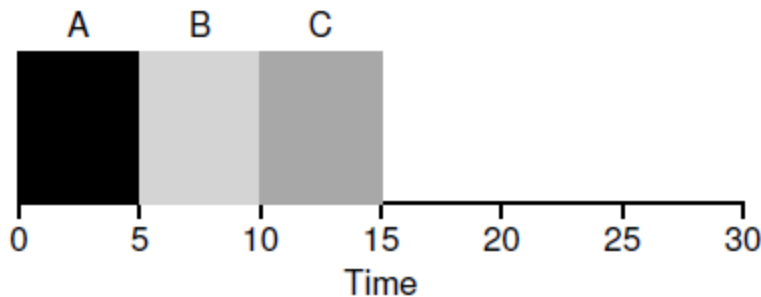
○ Round Robin

- Ne interesează **timpul de răspuns**
- 3 joburi A, B, C – se lansează deodată (în această ordine)
 - Fiecare rulează 5 secunde
 - Time-slice: 1 sec

○ Comparativ **SJF**

și

RR



○ Timpul de răspuns mediu

$$\frac{0+5+10}{3} = 5$$

$$\frac{0+1+2}{3} = 1$$

ALGORITMUL RR

- Amortizarea costurilor

- Schimbarea de context are un cost asociat
 - Schimbări mai rare -> eficiență mărită
- Ex. schimbarea de context durează 1ms și cuanta este 10 ms
 - 10 % din timp este pierdut pt. comutarea de la un job la altul
- Mărim cuanta de timp (time-slice) -> amortizarea costului
 - Ex. Cuanta 100 ms -> 1 % din timp folosit pt. comutare

- Durata cuantei

- Timp prea scurt
 - Scade eficiența
 - încărcare mare datorită schimbărilor de context frecvente
 - Deși performanța dată de metrica timpului de răspuns este mai bună
- Timp prea lung
 - Timpul de răspuns prea mare

- Compromis: 10 – 100 ms

ALGORITMUL RR

- Potrivit pentru sisteme unde metrica esențială este timpul de răspuns
- Politică mai corectă (fair)
 - Alocă procesorul în mod echitabil între joburi
- **Însă** - Timpul de deservire este foarte mare (14 pt exemplul precedent)
 - Cel mai slab algoritm în ceea ce privește timpul de finalizare
 - Pentru joburi lungi timpul de deservire poate deveni alarmant de mare
 - Poate chiar apare și înfometarea dacă procese noi continuă să intre frecvent în sistem

ALGORITMI DE PLANIFICARE

- Relaxăm încă o ipoteză – introducem op. de I/E

- ~~• Durata de execuție este aceeași pentru fiecare job~~

- ~~• Toate joburile sosesc în același moment în sistem~~

- ~~• Fiecare job folosește doar procesorul~~

- Durata de execuție este cunoscută în prealabil

- Decizie de planificare

- Când un job trebuie să aștepte pt operațiile de I/E

- Nu folosește procesorul -> se poate selecta alt job

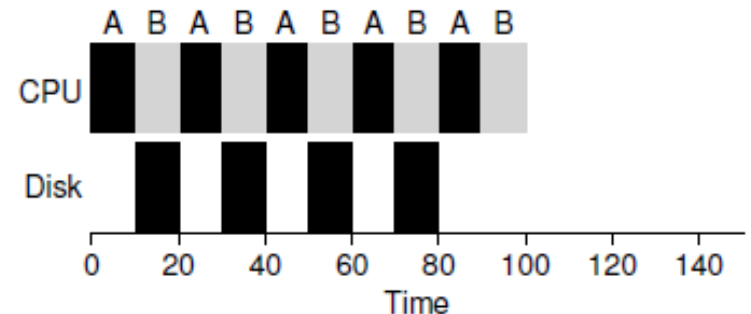
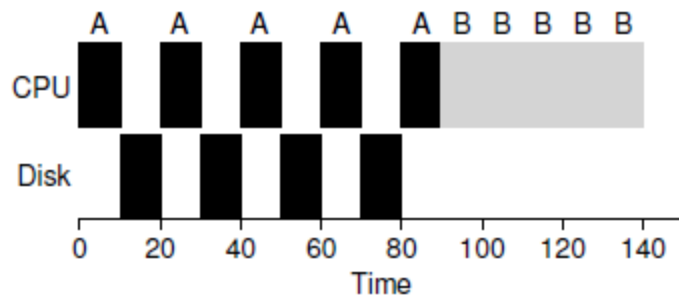
- Când operația de I/E este finalizată

- Procesul trece din starea Waiting în starea Ready

ALGORITMI DE PLANIFICARE

Exemplu

- Două joburi A și B – fiecare rulează 50 ms
 - A este lansat primul
 - după fiecare 10 ms de rulare apel I/E
 - Presupunem că operațiile de I/E durează 10 ms
 - B nu efectuează operații de I/E



- Utilizare inefficientă a resurselor
- Soluție
 - tratarea sub-joburilor din A ca joburi independente
 - Aplicare STCF -> suprapunere și intercalare

ALGORITMI DE PLANIFICARE

- Relaxăm încă o ipoteză – nu cunoaștem durata jobului
 - ◆ ~~Durata de execuție este aceeași pentru fiecare job~~
 - ◆ ~~Toate joburile sosesc în același moment în sistem~~
 - ◆ ~~Fiecare job folosește doar procesorul~~
 - ◆ ~~Durata de execuție este cunoscută în prealabil~~
- Ce facem?

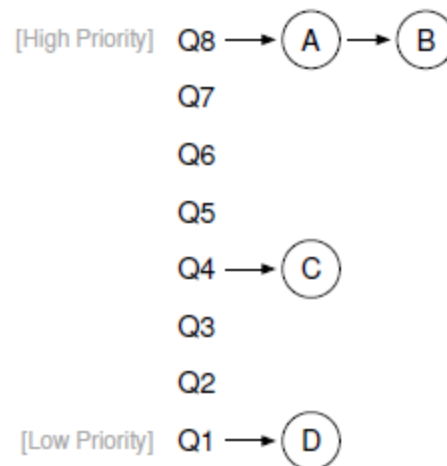
ALGORITMUL MLFQ

○ Multi-Level Feedback Queue

- Obiective
 - Minimizarea timpului de deservire
 - (SJF nu e aplicabil pt că nu știm durata unui job)
 - Minimizarea timpului de răspuns
 - importanța în sisteme interactive
 - (am văzut RR – dezavantajul timpului de deservire)
- În context realist – fără ipotezele de simplificare

ALGORITMUL MLFQ

- Coada de așteptare pentru procese Ready este divizată în mai multe cozi
 - Fiecare are un nivel diferit de prioritate
 - Se alege jobul cu cea mai mare prioritate
 - Dacă sunt mai multe - > RR în cadrul cozii



- Cum se asignează prioritățile?
 - Aspectul fundamental în MLFQ

ALGORITMUL MLFQ

○ Asignarea priorităților

- Dinamică – pe baza **comportamentului** observat
 - Învăță din trecut pentru a prezice viitorul
- Pt. joburi interactive se dorește menținerea unui nivel de prioritate ridicat
- Pt. joburile care folosesc intens procesorul prioritatea descrește treptat

○ Regula 1

- Dacă $Prioritate(A) > Prioritate(B)$: se alege A

○ Regula 2

- Dacă $Prioritate(A) = Prioritate(B)$ se aplică algoritmul RR între A și B

ALGORITMUL MLFQ

○ Cum se schimbă prioritatea?

- Cum se decide în care coadă trebuie să intre un job?
- Cum trece dintr-o coadă în alta?

○ Context

- Pe sistem avem
 - Joburi interactive, relativ scurte
 - Joburi intens computaționale
 - Timpul de răspuns nu este foarte important

○ Regula 3

- Când se lansează un job va fi pus în coada cu cea mai mare prioritate

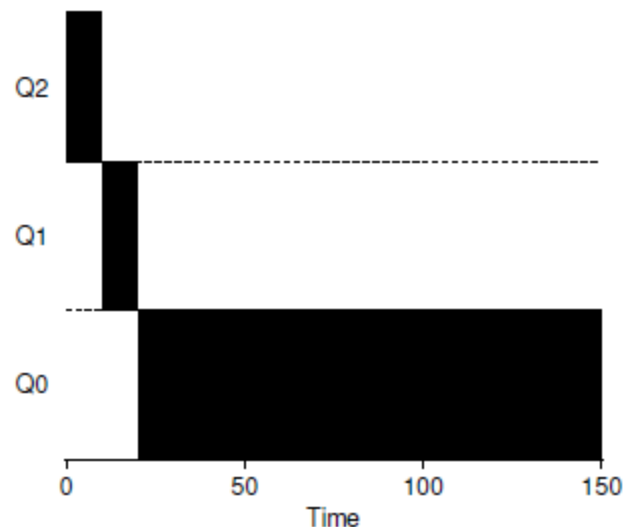
ALGORITMUL MLFQ

○ Regula 4a

- Dacă jobul își consumă cuanta de timp rulând -> se va reduce prioritatea

○ Regula 4b

- Dacă un job eliberează procesorul înaintea expirării cuantei de timp -> rămâne pe același nivel de prioritate



Long-running Job Over Time

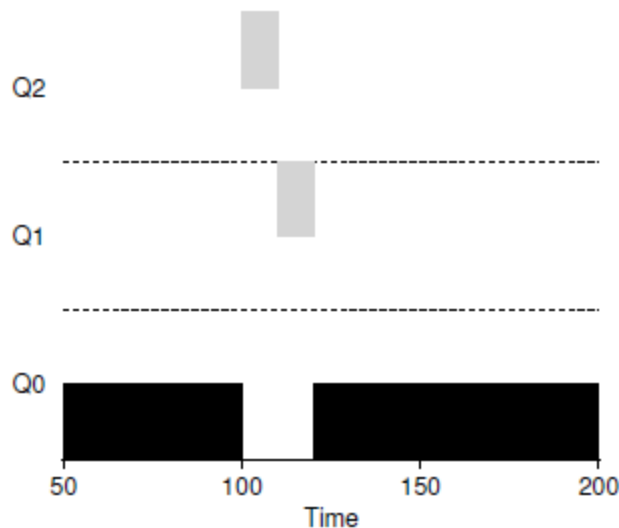
○ Exemplu pt.

- Un singur job lung
- Planificare pe 3 niveluri – 3 cozi
- Cuanta: 10 ms

ALGORITMUL MLFQ

- Apare un job mai scurt

- 2 joburi: A mai lung, rulează și apare B – scurt și interactiv
- B se termină în două cuante și A își reia execuția

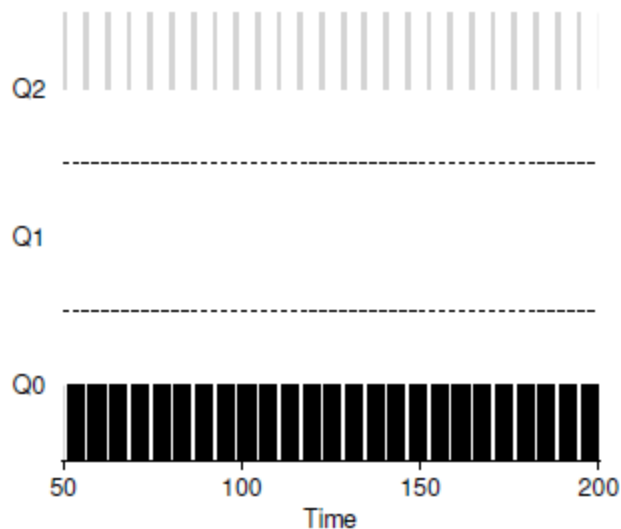


- B intră în coada cea mai prioritară
- Expira cuanta -> trece în coada următoare...

ALGORITMUL MLFQ

○ Introducem operațiile de I/E

- Regula 4 zicea că jobul care nu-și consumă cuanta rămâne la același nivel de prioritate
 - Avantaj joburi interactive – se execută rapid
- 2 joburi
 - A lung – gen batch
 - B interactiv folosește procesorul 1ms și apelează I/E



- B menținut la nivelul de prioritate mare

ALGORITMUL MLFQ

○ Probleme

- Deși pare destul de bun algoritmul
 - Joburile scurte și interactive se execută repede (timp de răspuns bun)
- Fenomenul de **înfometare**
 - Multe joburi interactive pot consuma tot timpul pe procesor
 - Joburile lungi nu apucă să se execute
- **Schimbarea comportamentului**
 - După o perioadă de calcule (prioritatea a scăzut treptat) devine interactiv – dar prioritatea nu mai crește

○ Soluție

- Amplificarea periodică a priorității tuturor joburilor din sistem

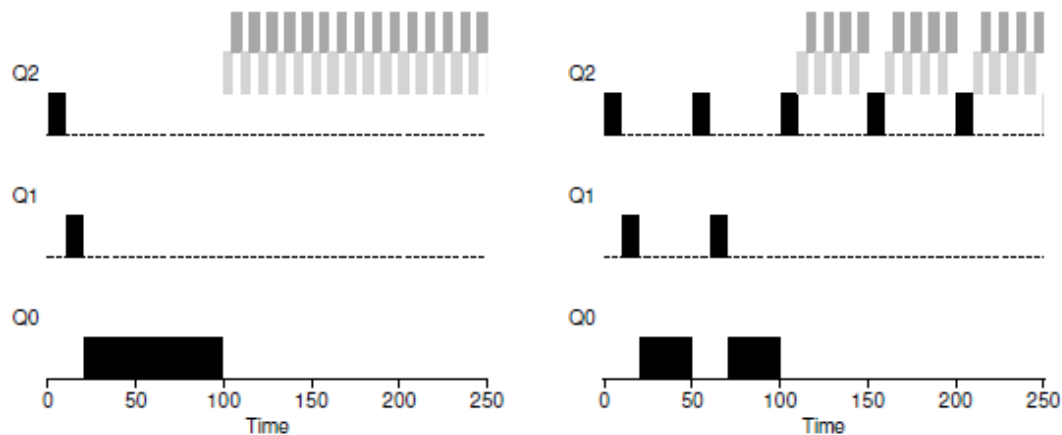
ALGORITMUL MLFQ

○ Regula 5

- După o perioadă S joburile reintră în coada cu cea mai mare prioritate

○ Soluționează ambele probleme

- Joburi lungi & Schimbare de comportament



Without (Left) and With (Right) Priority Boost

- Trei joburi
 - A lung
 - B, C scurte și interactive
- Perioada $S = 50$ ms

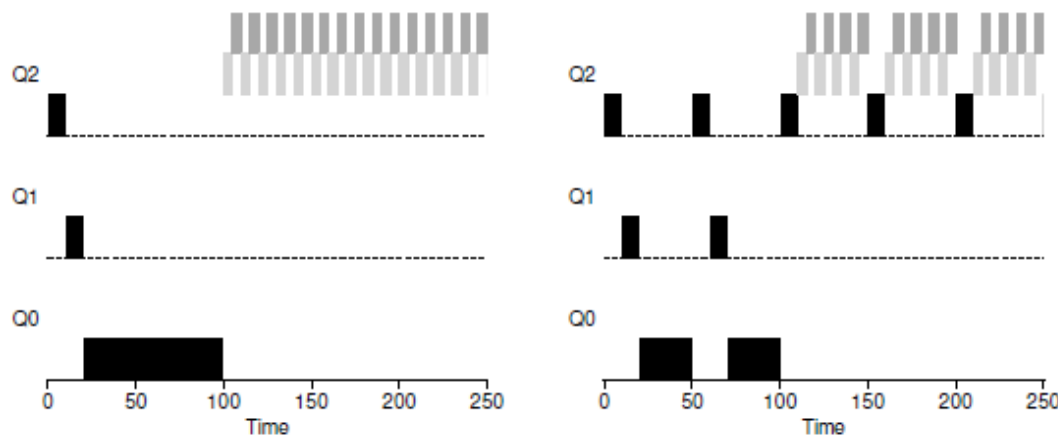
ALGORITMUL MLFQ

○ Regula 5

- După o perioadă S joburile reintră în coada cu cea mai mare prioritate

○ Soluționează ambele probleme

- Joburi lungi & Schimbare de comportament



Without (Left) and With (Right) Priority Boost

○ Cât să fie S -ul? (Constanta Voo-Doo)

- Prea mare $\rightarrow$ Joburile lungi sunt înfometate
- Prea mic $\rightarrow$ Joburile interactive nu răspund suficient de rapid

○ Trei joburi

- A lung
- B, C scurte și interactive

○ $S = 50$ ms

ALGORITMUL MLFQ

○ Problema

○ Dejucarea planificatorului

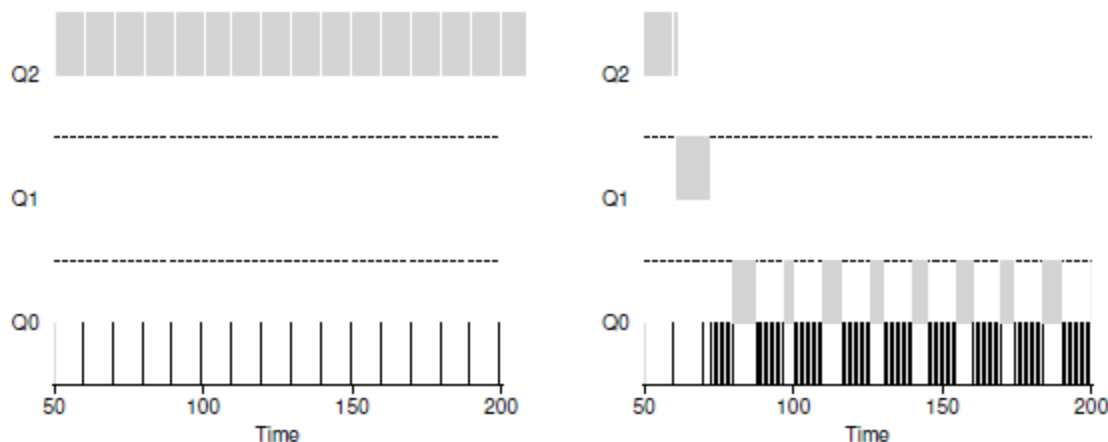
- Chiar înainte de expirarea cuantei apelează I/E pentru a rămâne pe același nivel de prioritate

○ Soluție – modificare Regula 4a și 4b

○ Regula 4

○ Prioritatea descrește la consumarea cuantei alocate

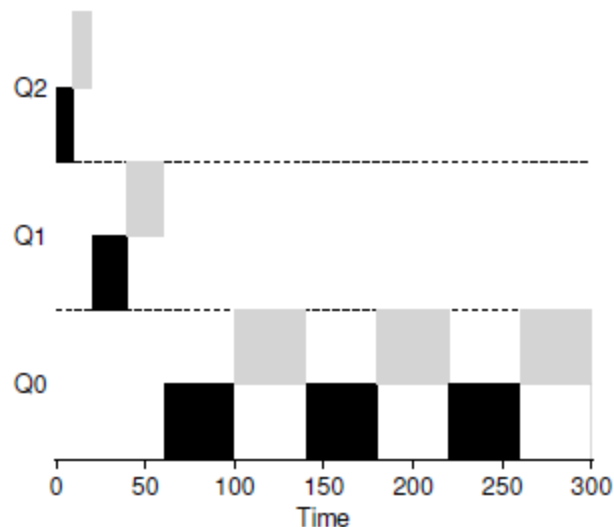
- indiferent dacă într-o singură rafală lungă sau mai multe scurte



ALGORITMUL MLFQ

○ Cum parametrizăm algoritmul?

- Câte cozi?
- Cât de mare e cuanta?
 - Cozile cu **prioritate mare** au **cuantă mică** (10 – 20 ms)
 - Cozile cu **prioritate mică** au **cuantă mai mare** (sute de ms)
- Cât de des să se facă amplificarea priorităților?



Lower Priority, Longer Quanta

PLANIFICARE PROPORȚIONALĂ

○ Scopul

- NU este să optimizăm timpii de deservire și de răspuns ci
- ESTE să **garantăm** că fiecare job obține un anumit procentaj din timpul procesorului - **echitabilitatea**

○ Planificare tip loterie

- Fiecare job primește un număr de bilete de loterie
 - Periodic (la expirarea cuantei de timp) se extrage un bilet câștigător în mod aleator - > determină jobul care va fi executat
 - Șansa de a fi ales este proporțională cu numărul de bilete
-
- Cum să asignăm biletele?
 - Joburile mai scurte primesc mai multe, cele lungi mai puține
 - Pentru a evita înfometarea, fiecare job primește cel puțin un bilet
 - Folosește aleatorismul pentru asigurarea corectitudinii
 - în mod probabilistic

PLANIFICARE TIP LOTERIE

- Ex. două joburi

- A are 75 de tichete (de la 0 la 74)
- B are 25 de tichete (de la 75 la 99)



- Loteria

- Numerele aleatoare din intervalul 0 – 99

63 85 70 39 76 17 29 41 36 39 10 99 68 83 63 62 43 0 49 49

- Planificarea corespunzătoare

A B A A B A A A A A A B A B A A A A A A

Table 9.4 Process Scheduling Example

Process	Arrival Time	Service Time
A	0	3
B	2	6
C	4	4
D	6	5
E	8	2

EXERCITIU

- Considerăm următoarele joburi

- FIFO sau FCFS**

- Non-preemptiv

Rulăm fiecare job în ordinea sosirii până se termină

Efectul de convoi: joburile scurte blocate în spatele celor mari

- Performanță mai bună pentru procese lungi
- Favorizează procesele intens computaționale

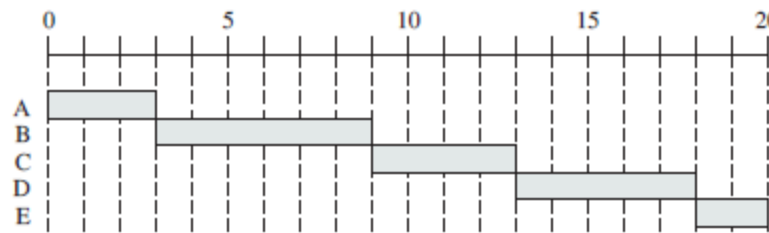


Table 9.4 Process Scheduling Example

Process	Arrival Time	Service Time
A	0	3
B	2	6
C	4	4
D	6	5
E	8	2

EXERCITIU

- Considerăm următoarele joburi

- SJF**

- Non-preemptiv
- Selectăm joburile cele mai scurte
- În caz real nu putem prevedea viitorul (durata jobului)
- Inechitabil (unfair) – procesele lungi pot fi înfometate

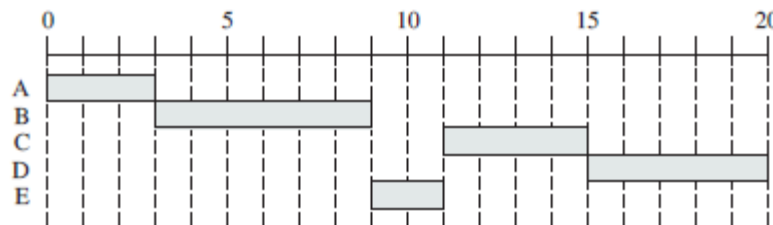


Table 9.4 Process Scheduling Example

Process	Arrival Time	Service Time
A	0	3
B	2	6
C	4	4
D	6	5
E	8	2

EXERCITIU

- Considerăm următoarele joburi

○ STCF

- Preemptiv - varianta preemptiva a SJF
- Selectăm jobul cu cel mai mic timp de execuție rămas
- Inechitabil (unfair) – procesele lungi pot fi înfometate
 - Joburile scurte sunt favorizate imediat ce apar

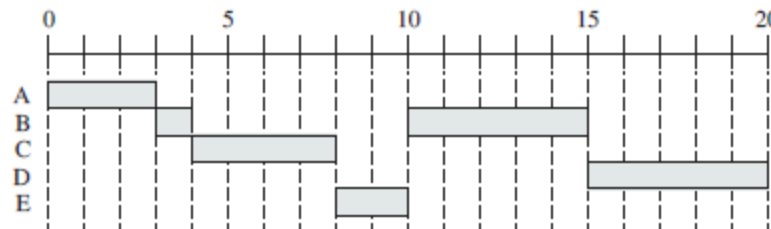


Table 9.4 Process Scheduling Example

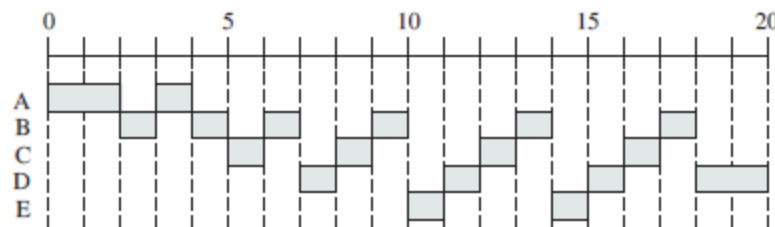
Process	Arrival Time	Service Time
A	0	3
B	2	6
C	4	4
D	6	5
E	8	2

EXERCITIU

○ Considerăm următoarele joburi

○ Round-Robin

- Preemptiv - time slice
- Trecem ciclic prin toate joburile
 - Avantaj pentru joburi scurte
 - Dezavantajos când joburile sunt de aceeași lungime
- Slice: 1



EXERCITIU

Table 9.4 Process Scheduling Example

Process	Arrival Time	Service Time
A	0	3
B	2	6
C	4	4
D	6	5
E	8	2

○ Round-Robin

- Preemptiv - time slice
- Trecem ciclic prin toate joburile
 - Avantaj pentru joburi scurte
 - Dezavantajos când joburile sunt de aceeași lungime
- Slice: 4

Daca cuanta depășește durata celui mai lung proces?

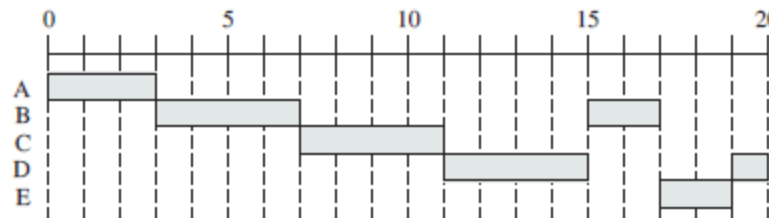


Table 9.4 Process Scheduling Example

Process	Arrival Time	Service Time
A	0	3
B	2	6
C	4	4
D	6	5
E	8	2

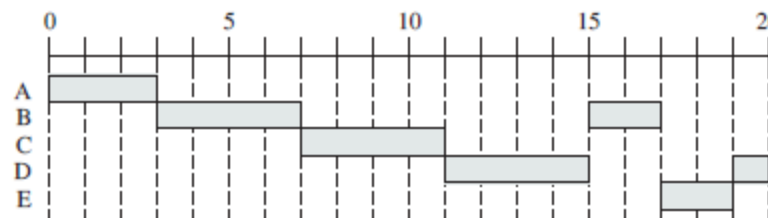
EXERCITIU

○ Considerăm următoarele joburi

○ Round-Robin

- Preemptiv - time slice
- Trecem ciclic prin toate joburile
 - Avantaj pentru joburi scurte
 - Dezavantajos când joburile sunt de aceeași lungime

• Slice: 4



Daca cuanta depășește
durata celui mai lung proces?
- RR devine similar cu FIFO

Table 9.4 Process Scheduling Example

Process	Arrival Time	Service Time
A	0	3
B	2	6
C	4	4
D	6	5
E	8	2

EXERCITIU

○ Considerăm următoarele joburi

○ MLFQ

- Preemptiv - cuanta
- La expirarea cuantei trece in coada mai puțin prioritară
- 5 reguli
- Performanță bună, echitabil

În aceste exemple se aplică MLFQ din momentul în care apare concurența

○ $q=1$ cuanta pe toate cozile este 1

○ $q=2^i$ unde i este numărul cozii

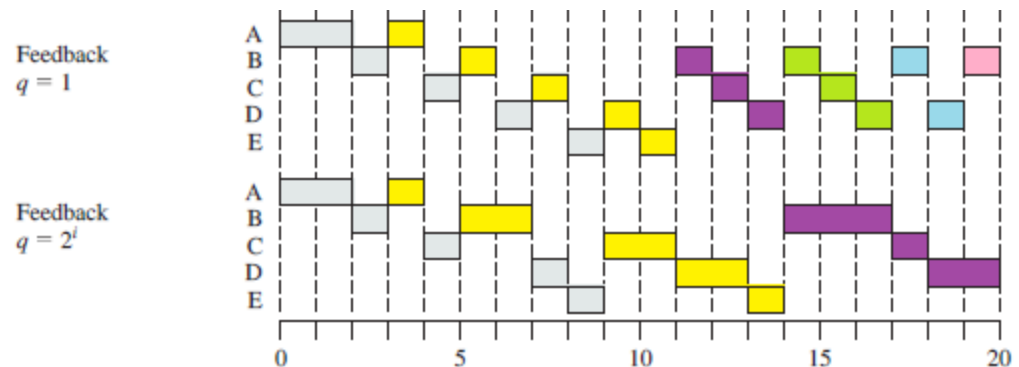


Table 9.5 A Comparison of Scheduling Policies

Process	A	B	C	D	E	
Arrival Time	0	2	4	6	8	
Service Time (T_s)	3	6	4	5	2	Mean
FCFS						
Finish Time	3	9	13	18	20	
Turnaround Time (T_r)	3	7	9	12	12	8.60
T_r/T_s	1.00	1.17	2.25	2.40	6.00	2.56
RR $q = 1$						
Finish Time	4	18	17	20	15	
Turnaround Time (T_r)	4	16	13	14	7	10.80
T_r/T_s	1.33	2.67	3.25	2.80	3.50	2.71
RR $q = 4$						
Finish Time	3	17	11	20	19	
Turnaround Time (T_r)	3	15	7	14	11	10.00
T_r/T_s	1.00	2.5	1.75	2.80	5.50	2.71
SJF						
Finish Time	3	9	15	20	11	
Turnaround Time (T_r)	3	7	11	14	3	7.60
T_r/T_s	1.00	1.17	2.75	2.80	1.50	1.84
STCF						
Finish Time	3	15	8	20	10	
Turnaround Time (T_r)	3	13	4	14	2	7.20
T_r/T_s	1.00	2.17	1.00	2.80	1.00	1.59
FB $q = 1$						
Finish Time	4	20	16	19	11	
Turnaround Time (T_r)	4	18	12	13	3	10.00
T_r/T_s	1.33	3.00	3.00	2.60	1.5	2.29
FB $q = 2^i$						
Finish Time	4	17	18	20	14	
Turnaround Time (T_r)	4	15	14	14	6	10.60
T_r/T_s	1.33	2.50	3.50	2.80	3.00	2.63

PLANIFICARE - REZUMAT

○ Planificare

- Selectarea unui job din coada Ready și alocarea procesorului

○ FCFS sau FIFO

- Rulăm fiecare job în ordinea sosirii până se termină
 - Simplu
 - Efectul de convoi: joburile scurte blocate în spatele celor mari

○ Shortest Job First / Shortest Time to Completion First

- Selectăm joburile cele mai scurte (non-preemptiv / preemptiv)
- Optim dacă metrica e timpul de execuție
- Dezavantaje
 - Nu putem prevedea viitorul (durata jobului)
 - Inechitabil (unfair)

PLANIFICARE - REZUMAT

○ Round Robin

- Alocăm fiecărui job o cuantă de execuție pe procesor
- Trecem ciclic prin toate joburile
 - Avantaj pentru joburi scurte
 - Dezavantajos când joburile sunt de aceeași lungime

○ Multi Level Feedback Queue

- Mai multe cozi cu priorități diferite
- Nu necesită cunoștințe a priori
- Învăță din comportamentele joburilor
 - Reguli pentru asignarea priorității
- Performanță bună, echitabil

○ Planificare tip loterie

- Joburile primesc un număr de bilete proporțional cu prioritatea lor
- Se alege aleator un bilet
- Implementare foarte simplă
- Echitabil
- Dezavantaj: problema alocării tichetelor

PLANIFICARE - REZUMAT

Table 9.3 Characteristics of Various Scheduling Policies

	FCFS	Round robin	SJF	STCF	Feedback
Selection function	max[w]	constant	min[s]	min[s - e]	(see text)
Decision mode	Non-preemptive	Preemptive (at time quantum)	Non-preemptive	Preemptive (at arrival)	Preemptive (at time quantum)
Throughput	Not emphasized	May be low if quantum is too small	High	High	Not emphasized
Response time	May be high, especially if there is a large variance in process execution times	Provides good response time for short processes	Provides good response time for short processes	Provides good response time	Not emphasized
Overhead	Minimum	Minimum	Can be high	Can be high	Can be high
Effect on processes	Penalizes short processes; penalizes I/O bound processes	Fair treatment	Penalizes long processes	Penalizes long processes	May favor I/O bound processes
Starvation	No	No	Possible	Possible	Possible

- w - timpul de așteptare
- e - timpul petrecut în execuție
- s – timpul total necesar execuției complete (inclus e)